Spark Assignment (Week 6):

Apache Spark Data Processing Assignment

This project demonstrates the implementation of an end-to-end data processing pipeline using Apache Spark (PySpark). The objective of the project was to understand Spark architecture, perform efficient data processing using DataFrames, and gain practical experience with Spark transformations, actions, schema handling, and optimized file formats.

The project began with setting up the PySpark environment and creating a Spark Session to initialize the Spark application. An employee dataset in CSV format was loaded into Spark using schema inference, allowing Spark to automatically identify the appropriate data types for each column. The structure of the dataset was examined using printSchema(), and the dataset contents were displayed using show() to verify successful loading.

Various DataFrame operations were then performed to understand Spark's data processing capabilities. Specific columns were selected to reduce unnecessary data processing, and filtering operations were applied to retrieve records satisfying given conditions such as age and payment tier. Existing columns were renamed to improve readability, data types were cast where necessary, and new columns were added using Spark functions to demonstrate DataFrame transformations. Duplicate records were removed, null values were handled using Spark's built-in functions, and sorting and grouping operations were performed to analyze the dataset.

The project also focused on understanding Spark's execution model. Lazy Evaluation was demonstrated by applying multiple transformations before triggering an action such as show() or count(). The concepts of Directed Acyclic Graph (DAG), lineage, transformations, and actions were studied to understand how Spark optimizes execution. Additionally, performance-related concepts including wide transformations, shuffle operations, predicate pushdown, and the advantages of using the Parquet file format over CSV were explored.

Finally, a complete ETL (Extract, Transform, Load) pipeline was built by reading the input CSV dataset, applying multiple transformations, filtering records, handling missing values, and writing the processed data into both CSV and Parquet formats. The project also followed Spark best practices such as avoiding unnecessary use of collect(), utilizing show() for displaying results, and selecting only the required columns to improve processing efficiency.

Overall, this assignment provided practical experience in distributed data processing using Apache Spark while demonstrating efficient data transformation techniques, schema management, optimized storage formats, and Spark performance optimization concepts.